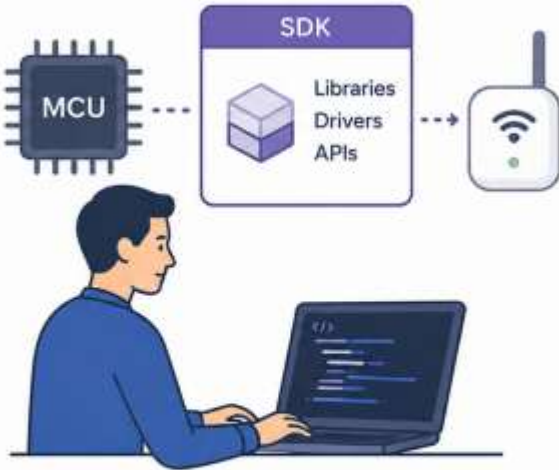




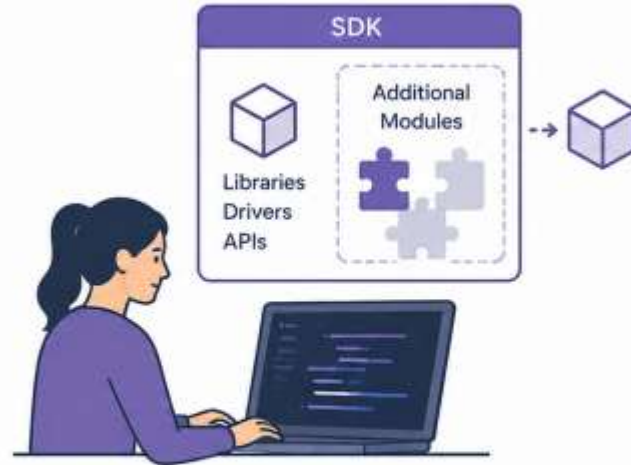
Two Developer Roles



Application Developer

Uses MCU and SDK to build IoT products

↓
prj.conf



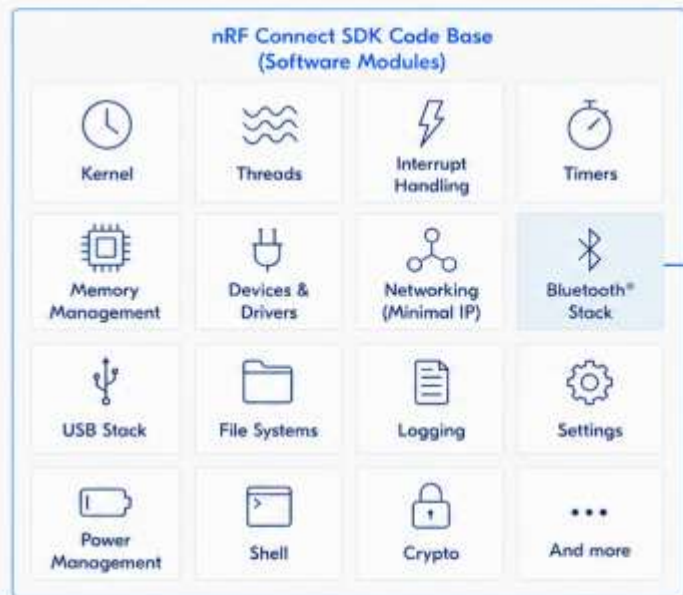
Platform Developer

Extends the SDK with additional software modules and capabilities

↓
Kconfig

nRF Connect SDK – Modular by design

nRF Connect SDK and Zephyr RTOS consist of many software modules. With **Kconfig**, you select exactly the modules and features you need and configure them for your application.



ENABLED MODULE
MODULE-SPECIFIC OPTIONS BECOME AVAILABLE



Only the modules and features you enable are included in your build.
Smaller footprint. Better performance. Tailored to your application.



Your optimized application

Using Kconfig Symbols in Configuration Files

SYNTAX

```
CONFIG_<symbol name>=<value>
```

Application configuration (e.g. prj.conf file)

```
SB_CONFIG_<symbol name>=<value>
```

Sysbuild configuration (e.g. sysbuild.conf file)

Kconfig supports these symbol types

bool true or false — value is y or n

int signed integer

hex hexadecimal integer

string text value

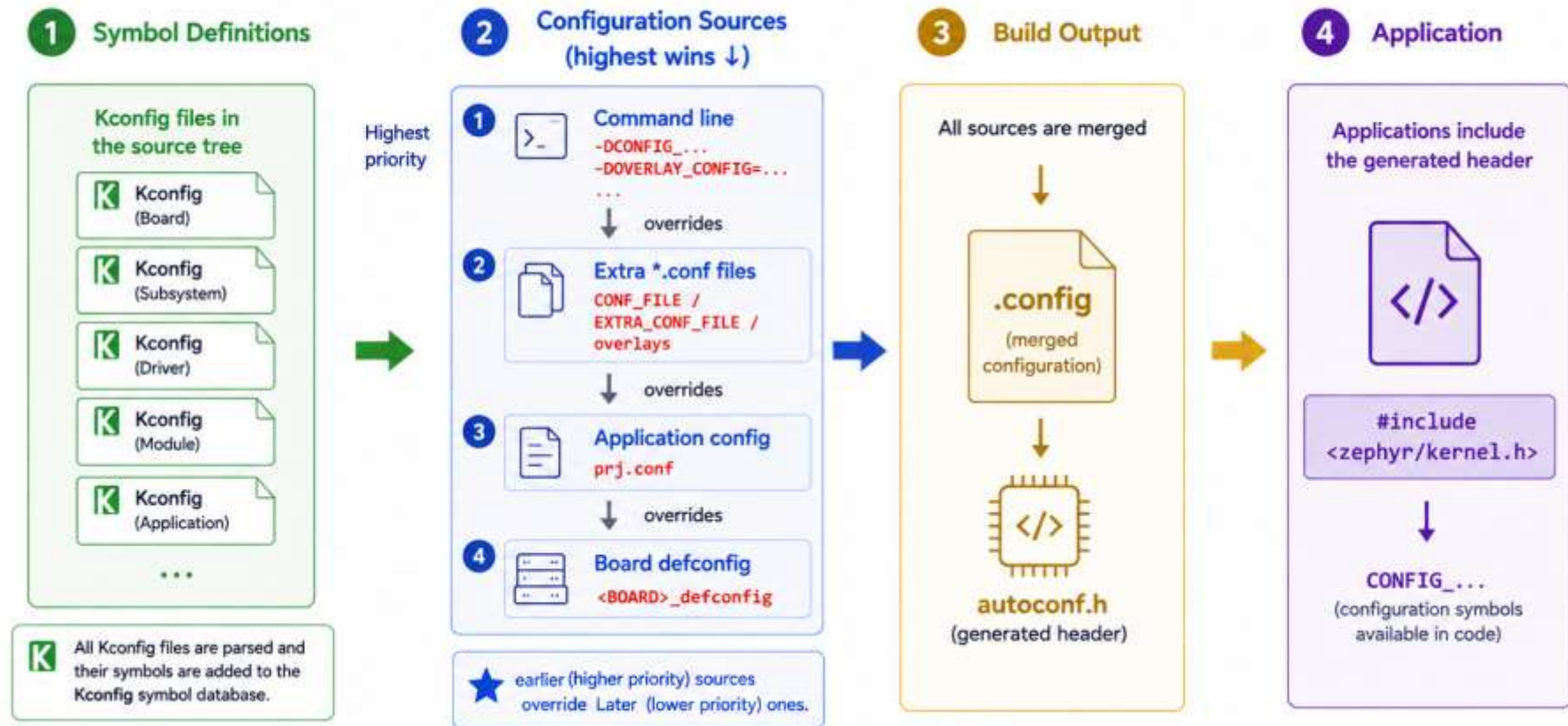
tristate — three-way boolean, value is y, n, or m (module) — mainly used in Linux Kconfig; Zephyr/nRF Connect SDK treats these effectively like **bool** since it has no loadable-module concept

COMMENTS

Comments use a "#". For example:

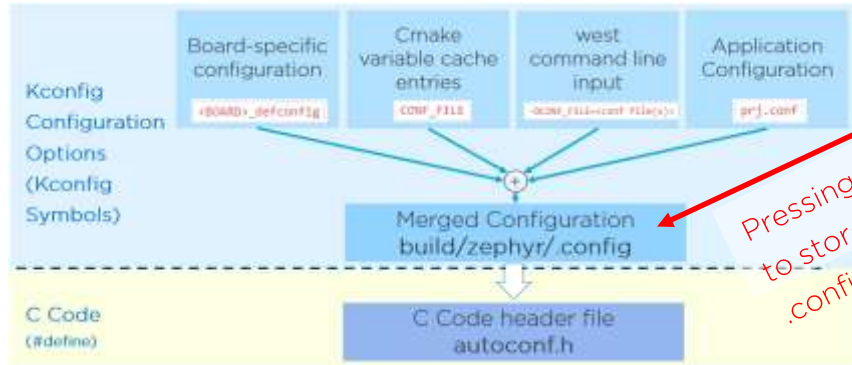
```
# This is a comment
```

How Kconfig Works: From Symbol Definitions to Application Code



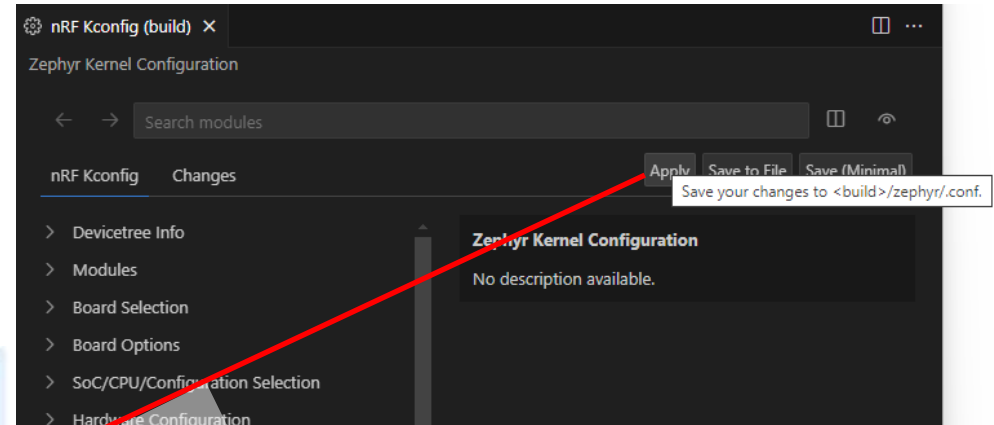
nRF Kconfig GUI

- Helpful tools:
 - nRF Kconfig GUI (default tool, included in VS Code environment)
 - menuconfig
 - *guiconfig*



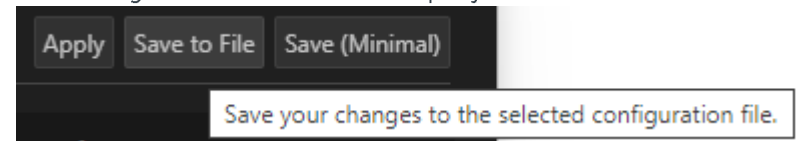
Pressing Apply causes
to store changes in
.config

IMPORTANT when using KCONFIG Tools:
A pristine build will erase the .config settings!
=> add CONFIG settings in prj.conf



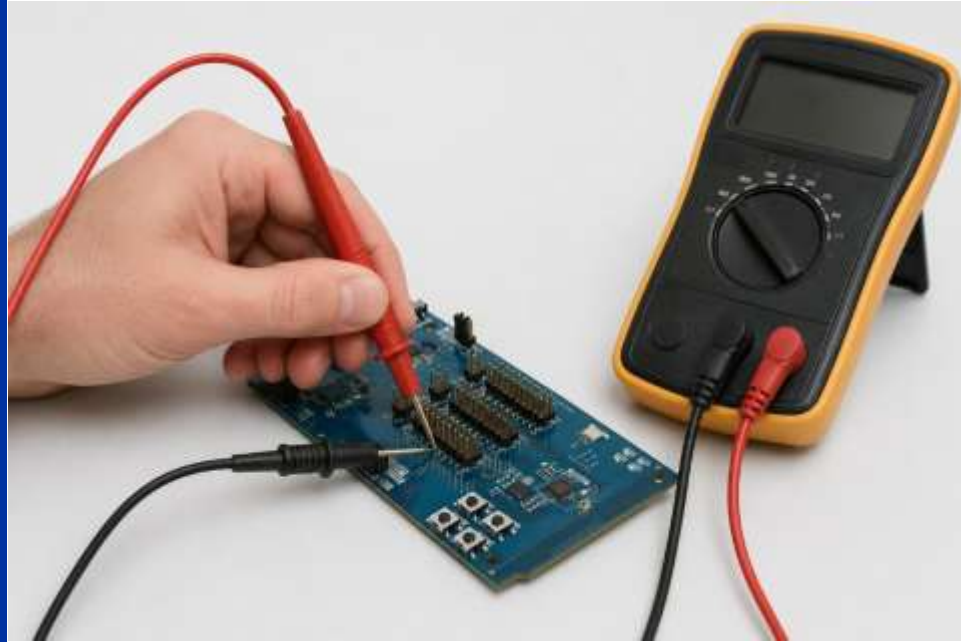
NOTE:

Pressing **Save to File** stores changes into your selected prj.conf file



Hands-on

- SYSBUILD
 - Adding MCUboot to our project



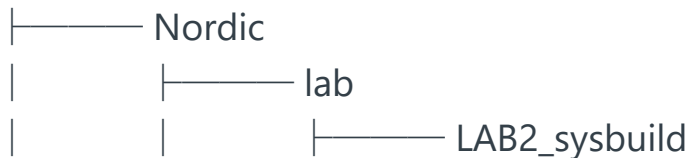


Step 1 - Copy Zephyr's *hello_world* sample

1. Create a new folder for your lab exercises.

For example: C:\Nordic\lab\ or on Linux/macOS: ~/lab/

2. Locate the **hello world** sample in your nRF Connect SDK installation: *install-path/zephyr/samples/hello_world*
3. Copy the entire **hello_world** directory into your lab workspace. Rename the **hello_world** project folder to: **LAB2_sysbuild**
4. Your workspace should now contain a structure similar to:





Step 2 - Open Project in VS Code

1. Open **Visual Studio Code**.
2. From the **nRF Connect** extension, select **Open an existing application**.
3. Browse to the copied *hello_world* / **LAB2_sysbuild** folder in your lab workspace and open it.
 **Tip:** Make sure you open the copied project, not the original sample located inside the SDK installation.

Step 3 - Include *MCUboot* in the multi-image build

7. Add a new file **sysbuild.conf** to your project folder and add following line in this file:

```
SB_CONFIG_BOOTLOADER_MCUBOOT=y
```



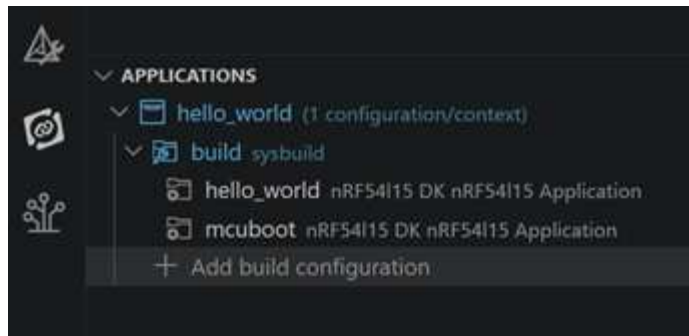
Step 4 - *Add Build Configuration* to your project and build the project

8. Ensure you are in the **nRF Connect** extension view.
9. In the **Applications** chapter click within the added project on **Add build configuration**.
10. Use following settings for the *build configuration*:
 - Board target: nrf54l15dk/nrf54l15/cpuapp
 - System build (sysbuild): Use sysbuild
11. Finally, click on the **Generate and Build** button.

Step 5 - Explore the Build Directory

12. In the **nRF Connect/Applications** view,
expand the build directory.

You should observe a structure similar to:



Examine each folder.



Step 6 - Changing MCUboot's Kconfig

13. Check to see if the following boot banner appears in the Serial Terminal.

```
*** Booting MCUboot v2.3.0-dev-c1d2d128a001 ***
```

```
*** Using nRF Connect SDK v3.4.0-99553055607b ***
```

```
*** Using Zephyr OS v4.4.0-bf801e4e3d19 ***
```

14. Let's disable the MCUboot boot banner. Create the following file in the specified directory:

C:/Nordic/lab/LAB2_sysbuild/**sysbuild/mcuboot.conf**

And add following Kconfig setting to this **mcuboot.conf** file:

```
CONFIG_MCUBOOT_BOOT_BANNER=n
```

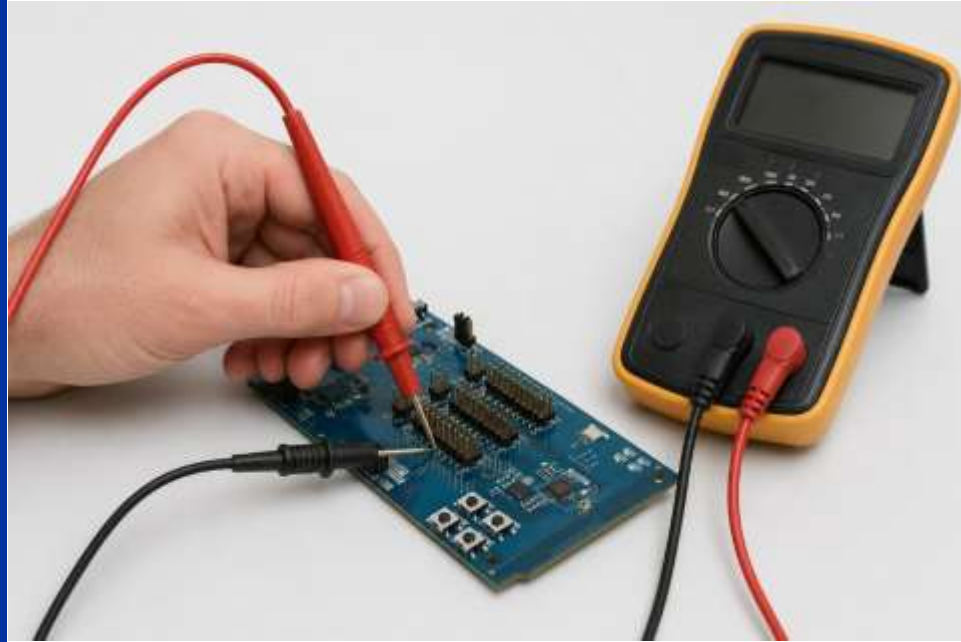
```
CONFIG_NCS_BOOT_BANNER=n
```

```
CONFIG_BOOT_BANNER=n
```

15. Build and flash the project. Then check the Serial Terminal output.

Hands-on

- KCONFIG
 - Add Zephyr Logging to our project
 - Using Segger Real Time Transfer (RTT) for Logging

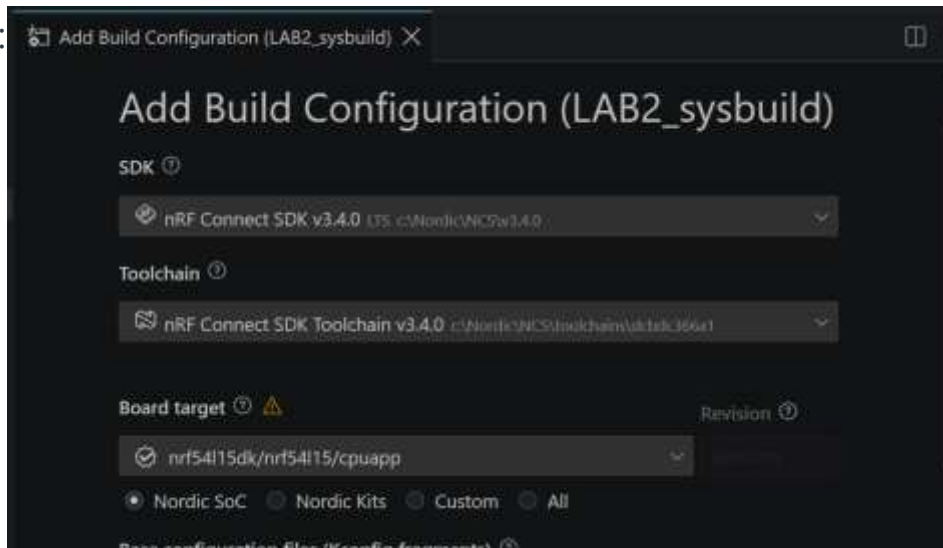


Step 1 — Create and Flash the hello_world Project

1. The **hello_world** sample ships with Zephyr and is included in every NCS checkout. Rather than building directly inside the SDK tree, copy it out into a workspace folder so you can edit it freely.

 **Tip:** Instead of manually copying the **hello world** project using File Explorer, you can also use the **Create a new application** -> **Copy a sample** feature in the **nRF Connect** view.

2. *Add build configuration* with following settings:



3. Flash the project and check Serial Terminal output.



Step 2 — Add Logging With the Default Configuration

4. Enable Logging subsystem by adding following line to **prj.conf** file: `CONFIG_LOG=y`
5. Register a Log module by adding following lines in **main.c** file:

```
#include <zephyr/kernel.h>
#include <zephyr/logging/log.h>
LOG_MODULE_REGISTER(app, LOG_LEVEL_DBG);
```

6. Add log calls in `main()` function:

```
LOG_ERR("This is an ERROR level log message");
LOG_WRN("This is a WARNING level log message");
LOG_INF("This is an INFO level log message");
LOG_DBG("This is a DEBUG level log message");
```

7. Build, flash and check Serial Terminal output.



Step 3 — Enable Logging commands in the Zephyr Shell

8. Update **prj_conf** and change logging configuration to:

```
CONFIG_LOG=y  
# Enable LOG commands in Shell  
CONFIG_LOG_CMDS=y
```

9. Rebuild, flash and check Serial Terminal
10. Why doesn't it work? What is the warning message in the terminal log?



Step 4 — Using the nRF Kconfig GUI to Trace Dependencies

💡 Manually editing `prj.conf` and grepping `.config` is a valid workflow, but it doesn't show you the *reasoning* behind a symbol — its help text, its depends on clause, what it selects, and what its default condition is. That reasoning is exactly what the **nRF Kconfig GUI**, built into nRF Connect for VS Code, is designed to surface.

11. Open *nRF Kconfig GUI* tool
12. Find `CONFIG_LOG_CMDS` and check its dependencies.
13. Which dependency is blocking here?
14. Fix it, rebuild, flash.
15. Is the `log` instruction available in the shell?

💡 *Tip:* Check by entering `help` in the Serial Terminal.



Step 5 — Add Logging via RTT

16. Let's add the snippet `rtt_console` to our **Build Configuration**.
17. Please disable the Zephyr **Shell** in the project.
18. Add a heart beat message to the application by adding following lines:

```
while(1) {  
    k_msleep(2000);  
    LOG_INF("heartbeat");  
}
```

19. Check the RTT output.